

Research Notes on Graph attention network

Graph attention network

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

>> Section 1

Graph neural networks are one of the main building blocks of AlphaFold, an artificial intelligence program developed by Google's DeepMind for solving the protein folding problem in biology. AlphaFold achieved first place in several CASP competitions.

>> Section 2

Social networks are a major application domain for GNNs due to their natural representation as social graphs. GNNs are used to develop recommender systems based on both social relations and item relations.

>> Section 3

Permutation equivariant: a permutation equivariant layer maps a representation of a graph into an updated representation of the same graph. In the literature, permutation equivariant layers are implemented via pairwise message passing between graph nodes. Intuitively, in a message passing layer, nodes update their representations by aggregating the messages received from their immediate neighbours. As such, each message passing layer increases the receptive field of the GNN by one hop. Local pooling: a local pooling layer coarsens the graph via downsampling. Local pooling is used to increase the receptive field of a GNN, in a similar fashion to pooling layers in convolutional neural networks. Examples include k-nearest neighbours pooling, top-k pooling, and self-attention pooling. Global pooling: a global pooling layer, also known as readout layer, provides fixed-size representation of the whole graph. The global pooling layer must be permutation invariant, such that permutations in the ordering of graph nodes and edges do not alter the final output. Examples include element-wise sum, mean or maximum. It has been demonstrated that GNNs cannot be more expressive than the Weisfeiler–Leman Graph Isomorphism Test. In practice, this means that there exist different graph structures (e.g., molecules with the same atoms but different bonds) that cannot be distinguished by GNNs. More powerful GNNs operating on higher-dimension geometries such as simplicial complexes can be designed. As of 2022, whether or not future architectures will overcome the message passing primitive is an open research question.

>> Section 4

$$\mathbf{h}_u = \phi\left(\mathbf{x}_u, \sum_{v \in N(u)} \psi(\mathbf{x}_u, \mathbf{x}_v, e_{uv})\right) \quad \{\displaystylestyle \mathbf{h}_u = \phi\left(\mathbf{x}_u, \sum_{v \in N(u)} \psi(\mathbf{x}_u, \mathbf{x}_v, e_{uv})\right) \}$$

>> Section 5

Local pooling layers Local pooling layers coarsen the graph via downsampling. Subsequently, several learnable local pooling strategies that have been proposed are presented. For each case, the input is the initial graph represented by a matrix $\mathbf{X}$ of node features, and the graph adjacency matrix $\mathbf{A}$. The output is the new matrix $\mathbf{X}'$ of node features, and the new graph adjacency matrix $\mathbf{A}'$.

>> Section 6

Scalability and distributed training Training GNNs on large graphs presents computational challenges that differ substantially from those in other neural network architectures. Two broad problem settings arise: training on a single very large graph (such as a social network with billions of nodes), and training on collections of large individual graphs (such as molecular systems requiring higher-order interaction modeling). For the single large graph setting, sampling-based methods reduce memory requirements by operating on subgraphs rather than the full graph. Cluster-GCN partitions the graph into subgraphs using graph partitioning algorithms, training the model on each partition in sequence. GraphSAINT instead samples subgraphs stochastically during training, constructing unbiased estimators for the full graph loss. Distributed frameworks such as DistDGL extend training to multi-machine settings, managing the inter-machine communication required for neighborhood aggregation when a graph's nodes are distributed across machines. For tasks such as atomic simulations, a different bottleneck arises: GNN architectures that model higher-order interactions between triplets or quadruplets of atoms are memory-intensive at the level of individual graphs, making standard data parallelism — where each GPU processes a separate sample — insufficient when individual graphs exceed single-device memory. Graph Parallelism addresses this by distributing a single input graph across multiple GPUs, partitioning nodes and edges across devices rather than distributing samples across devices.. This approach enabled GNNs with hundreds of millions to billions of parameters to be trained on atomic systems that would otherwise exceed single-device memory, and has been applied to the development of large-scale universal interatomic potentials.

>> Section 7

Research Notes on Graph attention network

Graph attention network

GNNs have become a primary tool for modeling atomic interactions in computational chemistry and materials science, where molecules and crystal structures are naturally represented as graphs with atoms as nodes and chemical bonds or spatial proximity as edges. A key advantage over traditional quantum chemistry methods such as density functional theory (DFT) is the reduction in computational cost from $O(n^3)$ to $O(n)$, enabling simulations at previously impractical scales. Early GNN architectures for this domain include SchNet (2017), which introduced continuous-filter convolutional layers for learning on 3D molecular structures, and DimeNet (2020), which incorporated angular information between atoms to improve geometric expressivity. Subsequent equivariant architectures such as NequIP and MACE, which respect the rotational and translational symmetries of physical systems, achieved significant accuracy improvements on standard benchmarks including QM9, MD17, and the Materials Project. Large open datasets have driven rapid progress in this area. The Open Catalyst 2020 (OC20) dataset, comprising over 260 million DFT calculations of adsorbate-catalyst surface interactions, established a widely used benchmark for energy and force prediction tasks relevant to catalyst discovery. The Materials Project and Alexandria databases have similarly provided large-scale crystal structure data for training universal interatomic potentials. More recent datasets including OMat24 and OMol25 have extended coverage to organic molecules and complex materials at greater computational scale. These datasets have enabled the development of universal [\[machine-learned interatomic potential|machine learning interatomic potentials\]](#) (MLIPs) — models trained across diverse chemical domains rather than on system-specific data. Examples include CHGNet, M3GNet, MACE-MP, and Meta FAIR's Universal Models for Atoms (UMA), the latter trained on approximately 500 million atomic structures spanning molecules, materials, and catalysts. GNoME, developed by Google DeepMind, applied GNNs to predict the stability of crystal structures, reporting the identification of millions of new stable candidate materials.

>> Section 8

where ϕ and ψ are differentiable functions (e.g., artificial neural networks), and $\bigoplus$ is a permutation invariant aggregation operator that can accept an arbitrary number of inputs (e.g., element-wise sum, mean, or max). In particular, ϕ and ψ are referred to as update and message functions, respectively. Intuitively, in an MPNN computational block, graph nodes update their representations by aggregating the messages received from their neighbours. The outputs

of one or more MPNN layers are node representations $\mathbf{h}_u$ for each node $u \in V$ in the graph. Node representations can be employed for any downstream task, such as node/graph classification or edge prediction. Graph nodes in an MPNN update their representation aggregating information from their immediate neighbours. As such, stacking n MPNN layers means that one node will be able to communicate with nodes that are at most n "hops" away. In principle, to ensure that every node receives information from every other node, one would need to stack a number of MPNN layers equal to the graph diameter. However, stacking many MPNN layers may cause issues such as oversmoothing and oversquashing. Oversmoothing refers to the issue of node representations becoming indistinguishable. Oversquashing refers to the bottleneck that is created by squeezing long-range dependencies into fixed-size representations. Countermeasures such as skip connections (as in residual neural networks), gated update rules and jumping knowledge can mitigate oversmoothing. Modifying the final layer to be a fully-adjacent layer, i.e., by considering the graph as a complete graph, can mitigate oversquashing in problems where long-range dependencies are required. Other "flavours" of MPNN have been developed in the literature, such as graph convolutional networks and graph attention networks, whose definitions can be expressed in terms of the MPNN formalism.